

МОЛДАВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Математики и Информатики

Департамент Информатики

По предмету Дискретная математика: теория графов.

Лабораторная работа № 1

Отчёт по программе: Поиск в глубину (DFS)

Лектор: N. Cuciuc

Выполнил: D. Belov

Кишинев, 2026

1. Теория и определения

Граф — это математическая структура, состоящая из:

- множества вершин
- множества рёбер, соединяющих пары вершин

В данной программе используется неориентированный граф — если есть ребро (u, v) , то можно двигаться и из u в v , и из v в u .

Неориентированный граф — это набор вершин, соединённых ребрами, не имеющими направления.

В программе граф представляется через список смежности вершин, реализованный в виде двумерного массива.

Две вершины называются **смежными**, если они соединены ребром.

Список смежности вершин — это способ представления графов, где каждая вершина хранит неупорядоченный список смежных с ней вершин.

Поиск в глубину (DFS) выполняется следующим образом:

- Начинаем с начальной вершины x , которая на данный момент является текущей вершиной.
- Вершина x посещается. Определяется её первый непосещённый сосед y , который становится текущей вершиной.
- Затем посещается первый непосещённый сосед y , и так далее, углубляясь, пока не достигнем вершины, у которой больше нет непосещённых соседей. Когда достигаем такой вершины, возвращаемся к её «родительской» вершине - той, из которой мы пришли.
- Если у этой вершины ещё есть непосещённые соседи, выбираем следующего непосещённого и продолжаем обход тем же способом.
- Если у вершины больше нет непосещённых соседей, возвращаемся к её родительской вершине и продолжаем тем же способом, пока не будут посещены все вершины, доступные из стартовой вершины.

2. Описание алгоритма и принцип работы

Алгоритм работает следующим образом:

- 1 Пользователь вводит:
 - количество вершин
 - количество рёбер
 - перечень рёбер
- 2 Формируется список смежности вершин
- 3 Пользователь вводит стартовую вершину
- 4 Запускается рекурсивная функция GrafCalc

5 Алгоритм:

- помечает вершину как посещённую
- добавляет её в массив ответа
- рекурсивно вызывает себя для смежных вершин

Разбор функции GrafCalc

```
void GrafCalc(int nod, int arr[][9],  
    int ans[], bool pointarr[], int actual,  
    int target)
```

Параметры:

nod — количество вершин

arr — список смежности вершин

ans — массив результата (порядок обхода)

pointarr — массив посещённых вершин

actual — текущая вершина

target — следующая вершина

Логика работы

1 Перебираются все смежные вершины для текущей вершины:

```
for (int i = 0; i < nod; i++)
```

2 Если вершина:

существует (next != 0)

ещё не посещена (pointarr[next - 1] == false)

3 Тогда:

помечаем её посещённой

добавляем в ответ

вызываем рекурсию

3. Примеры применения DFS

1. Проверка связности графа

DFS позволяет определить:

- связан ли граф
- существуют ли изолированные вершины

2. Определение возможности пути между вершинами

Можно определить, достижима ли одна вершина из другой.

Конкретный прикладной пример использования

Поиск в глубину (DFS) широко применяется при анализе компьютерных сетей для проверки связности и выявления изолированных сегментов. Если представить устройства как вершины графа, а соединения между ними как рёбра, то запуск DFS из

одного узла позволяет определить все устройства, которые доступны из него напрямую или через другие узлы. Таким образом можно проверить, является ли сеть полностью связной, или в ней существуют отдельные компоненты. Это особенно важно при диагностике инфраструктуры, когда необходимо обнаружить отключённые устройства, ошибки конфигурации или разрывы соединений.

Пример работы программы

- Вход:

Количество вершин: 5

Количество рёбер: 4

Рёбра:

1 2

1 3

2 4

3 5

Стартовая вершина: 1

- Граф:

1 — 2 — 4

|

3 — 5

- Результат обхода:

1 → 2 → 4 → 3 → 5

4. Выводы и оценка сложности

- Временная сложность

В данной программе перебор соседей идёт через массив фиксированного размера 9×9 , поэтому фактически сложность $\approx O(n^2)$

- Пространственная сложность

Используется:

массив смежности $9 \times 9 \rightarrow O(V^2)$

массив посещённых $\rightarrow O(V)$

массив ответа $\rightarrow O(V)$

стек рекурсии $\rightarrow O(V)$

Итого: $O(V^2)$

- Ограничения программы

- 1) максимум 9 вершин (технически легко реализовать, ограничение поставлено для сохранения визуального порядка при выводе)
- 2) нет проверки на повторный ввод рёбер (в программе стоит предупреждение)
- 3) используется фиксированный размер массива (не используется динамическая память)

- Выводы

В работе был рассмотрен алгоритм поиска в глубину, который был реализован через рекурсию. Была проанализирована сложность, описаны ограничения программы, а также её пример использования.

Алгоритм корректно выполняет обход графа в глубину и демонстрирует базовые принципы рекурсивного обхода структур данных.

5. Листинг программы

```
#include <iostream>
using namespace std;

void GrafCalc(int nod, int arr[][9],
int ans[], bool pointarr[], int actual,
int target) {
    for (int i = 0; i < nod; i++) {
        int next = arr[actual - 1][i];
        if (next != 0 && pointarr[next - 1] == false) {
            pointarr[next - 1] = true;
            for (int j = 0; j < nod; j++) {
                if (ans[j] == 0 && next != 0) {
                    ans[j] = next;
                    break;
                }
            }
            target = next;
            GrafCalc(nod, arr, ans, pointarr, target, target);
        }
    }
}

int main()
{
    int numberOfDots = 0, numberOfRibs = 0, startPoint = 1;
    int array[9][9] = { 0 }, answerArr[9] = { 0 };
    bool pointedArr[9] = { 0 };
```

```

cout << "How many points do you have? (max 9)\n";
cin >> numberOfDots;
cout << "How many ribs do you have? (max 20)\n";
cin >> numberOfRibs;

cout << "Enter the ribs in format: point1 point2\n";
for (int i = 0; i < numberOfRibs; i++)
{
    int point1 = 0, point2 = 0;
    cin >> point1 >> point2;

    if (point1 < 1 || point1 > numberOfDots ||
        point2 < 1 || point2 > numberOfDots)
    {
        cout << "Invalid!\n";
        i--;
        continue;
    }

    for (int j = 0; j < numberOfDots; j++) {
        if (array[point1 - 1][j] == 0) {
            array[point1 - 1][j] = point2;
            break;
        }
    }
    for (int j = 0; j < numberOfDots; j++) {
        if (array[point2 - 1][j] == 0) {
            array[point2 - 1][j] = point1;
            break;
        }
    }
}

for (int i = 0; i < numberOfDots; i++) {
    cout << i + 1 << " || ";
    for (int j = 0; j < numberOfDots; j++) {
        if (array[i][j]) { cout << array[i][j] << " | "; }
    }
    cout << endl;
}

cout << "Enter the start point:\n";
cin >> startPoint;

answerArr[0] = startPoint;
pointedArr[startPoint - 1] = true;
GrafCalc(numberOfDots, array, answerArr, pointedArr, startPoint, startPoint);

```

```
cout << "way:\n" << "start -- > ";  
for (int i = 0; i < numberOfDots; i++) {  
    cout << answerArr[i] << " --> ";  
}  
cout << "finish";  
}
```